

System Architecture Design (SAD)

Offline Gesture Recognition Application, Version 1

1. Purpose and Scope

This document defines the high-level software architecture for Version 1 of a cross-platform desktop application that recognizes ten predefined hand gestures in real time using a local camera. The application is developed in Python, uses PySide6 for the graphical user interface (GUI), and uses OpenCV for camera capture and image processing.

The system must operate entirely offline on Windows, macOS, and Linux. Camera frames and recognition data are processed in memory and are not collected, persistently stored, or transmitted. Local configuration and diagnostic logs are permitted, provided they exclude camera frames and sensitive user data.

2. Architectural Overview

The application follows a modular, single-process, multithreaded architecture. Components communicate through defined interfaces and shared domain data contracts. The Qt main thread manages the GUI, while camera capture and computationally intensive vision processing execute in worker threads.

The recognition engine uses pretrained hand landmark detection combined with a hybrid recognition layer. Geometric rules handle suitable static poses, temporal state machines detect directional movements, and machine-learning classifiers are used where appropriate. A centralized decision and event management component validates recognition results, applies confidence thresholds, and suppresses duplicate events.

All models and runtime dependencies must be available locally. Runtime network access and automatic model downloads are excluded from the design.

High-level architecture diagram

flowchart TD

```
UI["Presentation Layer<br/>PySide6 GUI"]
AC["Application Controller"]
CAM["Camera Module<br/>Capture and Timestamps"]
VISION["Vision Module<br/>Hand Detection and Landmarks"]
TRACK["Tracking and Feature Module<br/>Motion and Spatial Features"]
REC["Gesture Recognition Module"]
STATIC["Static Pose Recognizer"]
TEMP["Temporal Movement Recognizer"]
TWO["Two-Hand Recognizer"]
DEC["Recognition Decision and Event Manager"]
CFG["Configuration Module"]
LOG["Logging and Error Management"]
DOMAIN["Domain Module<br/>Data Contracts and States"]
```

```
UI <--> AC
AC <--> CAM
CAM --> VISION
VISION --> TRACK
TRACK --> REC
```

```

REC --> STATIC
REC --> TEMP
REC --> TWO
STATIC --> DEC
TEMP --> DEC
TWO --> DEC
DEC --> AC
AC --> UI
CFG --> AC
CFG --> DEC
DOMAIN -. "Shared contracts" .-> AC
DOMAIN -. "Shared contracts" .-> VISION
DOMAIN -. "Shared contracts" .-> REC
DOMAIN -. "Shared contracts" .-> DEC
AC --> LOG
CAM --> LOG
VISION --> LOG
REC --> LOG
DEC --> LOG

```

Diagram description for document generation: Use rectangular component boxes with directional arrows to represent processing flow and control communication. Place the Presentation Layer at the top, the Application Controller beneath it, and the camera, vision, tracking, and recognition components along the main processing path. Show the three recognizers as parallel branches converging on the Recognition Decision and Event Manager. Place Configuration and Domain modules as supporting components, with Logging and Error Management connected to relevant operational components. Use dashed arrows for shared domain contracts.

3. Major Components and Responsibilities

Component	Responsibility
Presentation Layer	Displays the camera preview, bounding boxes, recognized gesture labels, and five indicators: Last Gesture, Current Gesture, Confidence, Camera FPS, and Processing FPS.
Application Controller	Coordinates application startup, shutdown, worker lifecycle, component communication, and recognition state updates.
Camera Module	Initializes the camera, captures timestamped frames, and reports device or capture failures.
Vision Module	Performs hand detection, landmark extraction, and required image preprocessing.
Tracking and Feature Module	Maintains hand tracking, derives finger-pose features, tracks motion history, and calculates two-hand spatial relationships.
Gesture Recognition Module	Provides replaceable recognition strategies for static poses, temporal movements, and two-hand gestures.
Recognition Decision and Event Manager	Applies confidence thresholds, validates recognition criteria, manages re-arming and duplicate suppression, and emits recognition events.
Configuration Module	Loads, validates, and supplies application settings, including the default confidence threshold of 0.7.
Logging and Error Management Module	Records operational events, diagnostic information, exceptions, and recovery actions without sensitive image data.
Domain Module	Defines shared data contracts, gesture identifiers, recognition results, application states, and error types.

The ten externally recognized gesture labels are Swipe Left, Swipe Right, Peace Sign, Close Hand, Open Hand, Wave Left with Index Finger Raised, Wave Right with Index Finger Raised, Wave Left with Index and Middle Fingers Raised, Wave Right with Index and Middle Fingers Raised, and Join Both Hands in a Clap/Prayer Gesture.

The static prayer pose and dynamic clap motion must produce the same external gesture label. Internal recognition strategies may distinguish these patterns while exposing a consistent result through the recognition interface.

4. Runtime Data Flow

1. The Application Controller initializes the configuration, loads local recognition models, and starts the camera and processing workers.
2. The Camera Module captures frames and timestamps them. A bounded buffer uses a latest-frame-wins policy to discard obsolete frames when processing cannot keep pace with capture.
3. The Vision Module detects hands and extracts landmarks. The Tracking and Feature Module derives pose, motion, and two-hand relationship features.
4. The Gesture Recognition Module evaluates applicable static, temporal, and two-hand recognition strategies. Candidate results include a gesture identifier, confidence score, and relevant tracking information.
5. The Recognition Decision and Event Manager validates the candidate against gesture-specific criteria and the configured confidence threshold. The default threshold is 0.7, with additional temporal validation where required.
6. Upon a valid recognition, the manager emits an event to the Application Controller, which updates the GUI through thread-safe communication. Recognized hands receive green bounding boxes and gesture labels; hands without an active recognized gesture receive grey bounding boxes.
7. The GUI updates its five indicators using the latest valid recognition and performance measurements. Re-arming, cooldown, or gesture-specific state management prevents repeated events for sustained gestures.

The GUI must remain responsive throughout processing. UI updates occur exclusively on the Qt main thread.

5. Key Architectural Decisions and Trade-offs

Modular, single-process execution: A single process simplifies deployment and component integration. Worker threads isolate blocking capture and expensive processing from the GUI, although Python runtime constraints and model-library behavior must be considered when selecting concurrency mechanisms.

Pretrained and hybrid recognition: Pretrained landmark detection reduces the need to develop a hand detector from scratch. Geometric rules are interpretable and efficient for suitable static poses; temporal state machines represent directional movements; classifiers can accommodate patterns that

are difficult to express reliably through rules alone. This hybrid design improves flexibility but introduces model dependencies and additional validation requirements.

Replaceable recognition interface: The controller communicates with the recognition engine through a stable interface, allowing models and classification strategies to change without redesigning the camera or presentation components.

Latest-frame-wins buffering: A bounded frame queue prioritizes current visual information and limits latency and memory growth. Frames may be dropped under load, trading processing completeness for responsiveness.

Confidence and event control: A configurable default threshold of 0.7 provides a common baseline. Gesture-specific temporal criteria and duplicate suppression address false activations and repeated events. Confidence calibration and gesture-specific criteria require empirical validation.

Offline operation: Bundled or locally installed models and dependencies avoid runtime network requirements and reduce data exposure. The trade-off is that dependency installation, model availability, and version management must be handled explicitly during development and deployment.

6. Error Handling, Logging, and Privacy

Errors are handled at component boundaries and reported to the Application Controller for coordinated recovery or controlled shutdown. Expected failure cases include unavailable cameras, capture interruptions, model-loading failures, inference exceptions, invalid configuration, worker-thread failures, and shutdown errors.

Recoverable failures should trigger appropriate retries or component restarts where safe. Unrecoverable failures should produce a clear user-facing error and release camera and worker resources cleanly. Logging records lifecycle events, error categories, component status, and recovery actions while excluding image data, landmarks that could expose user activity, and other sensitive information.

Camera frames and transient recognition data remain in memory. No cloud services, remote APIs, telemetry, databases, or network-dependent runtime features are introduced.

7. Extensibility and Testing

Clear interfaces and shared domain contracts support the addition of gestures, replacement of recognition models, and adjustment of recognition policies without coupling those changes to the GUI or camera implementation.

Testing includes unit tests for geometric features, recognition state machines, confidence enforcement, event suppression, and configuration validation; integration tests for camera-to-recognition data flow and UI event delivery; and hardware-dependent tests for camera access, performance, and cross-platform behavior. Model-loading failures, dropped frames, concurrent shutdown, and worker exceptions must also be tested.

The architecture and technology stack are established decisions. The exact landmark model, classifier selection, gesture-specific temporal parameters, thread implementation, camera selection policy, and performance targets remain implementation decisions to be finalized through prototyping and testing. Software Design Plan (SDP)

Offline Gesture Recognition Application, Version 1

Document status: Draft | Based on the approved System Architecture Design (SAD)

1. Introduction

1.1 Purpose

This Software Design Plan (SDP) defines the software design for Version 1 of the Offline Gesture Recognition Application. It translates the System Architecture Design into a component-level design, specifying module interfaces, internal responsibilities, data structures, processing logic, concurrency, error handling, and testing requirements.

1.2 Scope

The application is a cross-platform desktop application developed in Python using PySide6 for the graphical user interface and OpenCV for camera capture and image processing. It recognizes ten predefined hand gestures in real time using locally available pretrained models and a hybrid recognition approach combining geometric rules, temporal state machines, and machine-learning classifiers.

The application supports Windows, macOS, and Linux and must operate without network access. Camera frames and transient recognition data are processed in memory and are not persistently stored or transmitted.

1.3 Design Constraints

The design shall adhere to the following constraints:

- The application shall use a modular, single-process, multithreaded architecture.
- GUI operations shall execute exclusively on the Qt main thread.
- Camera capture and computationally intensive vision processing shall execute in worker threads.
- Recognition models and runtime dependencies shall be available locally.
- A bounded latest-frame-wins buffer shall prioritize processing current camera frames.
- The default recognition confidence threshold shall be 0.7.
- The static prayer pose and dynamic clap motion shall produce the same external gesture label.
- Configuration and diagnostic logging shall exclude camera frames and sensitive user data.

2. Software Design Overview

The software is divided into presentation, application coordination, camera acquisition, vision processing, tracking and feature extraction, gesture recognition, recognition decision management, configuration, logging, and domain modules.

Each module shall expose a defined interface and minimize dependencies on other modules' internal implementations. Shared data structures shall be defined in the Domain Module. The Application Controller shall coordinate module lifecycles and data exchange without implementing vision algorithms or gesture-specific recognition logic.

2.1 Design Structure

Figure SDP-01: Software Module Decomposition Diagram

Diagram label: SDP-01 – Software Module Decomposition

Design specification: Represent the ten software modules as rectangular boxes grouped into three logical areas: presentation and application coordination; camera and recognition processing; and supporting services and shared domain definitions. Show the primary processing dependencies from camera acquisition through recognition and decision management to GUI updates. Represent Configuration and Logging as supporting services, and Domain as the provider of shared data contracts.

2.2 Design Principles

The implementation shall follow separation of concerns, interface-based communication, replaceable recognition strategies, explicit application states, bounded buffering, and deterministic resource cleanup. Recognition algorithms shall remain independent of GUI widgets and camera-specific APIs. Thread communication shall use Qt signals and slots or equivalent thread-safe mechanisms supported by the selected runtime.

3. Module Design

3.1 Presentation Module

The Presentation Module shall implement the desktop interface using PySide6. It shall display the live camera preview, hand bounding boxes, recognized gesture labels, and the following performance and recognition indicators:

- Last Gesture
- Current Gesture
- Confidence
- Camera FPS
- Processing FPS

Recognized hands shall be displayed with green bounding boxes and associated gesture labels. Hands without an active recognized gesture shall be displayed with grey bounding boxes.

The module shall emit user actions, such as application start and stop requests, to the Application Controller. It shall receive recognition results, camera status, and performance updates through thread-safe signals or slots. It shall not perform inference, direct camera capture, or other computationally intensive operations.

3.2 Application Controller Module

The Application Controller shall coordinate initialization, execution, state transitions, and shutdown. It shall initialize configuration and recognition services, load local models, create and start worker threads, and connect component interfaces.

The controller shall receive validated recognition events and publish updates to the Presentation Module. It shall also coordinate recovery from component failures and ensure that camera and worker resources are released during shutdown.

The controller shall maintain the application lifecycle states:

- Initializing: Configuration and required resources are being prepared.
- Ready: Required components are initialized and available.
- Running: Camera capture and recognition are active.
- Error: A component failure requires recovery or user notification.
- Stopping: Workers and resources are being shut down.
- Stopped: Execution has terminated and resources have been released.

Figure SDP-02: Application Lifecycle State Machine

Diagram label: SDP-02 – Application Lifecycle State Machine

Design specification: Represent lifecycle states as state-machine nodes. Show initialization leading to Ready, Ready leading to Running, Running leading to Stopping, and Stopping leading to Stopped. Include error transitions from initialization and runtime states, with recovery paths where appropriate and controlled shutdown for unrecoverable failures.

3.3 Camera Module

The Camera Module shall encapsulate OpenCV camera access. It shall initialize the selected camera, acquire frames, assign capture timestamps, and report capture status or device errors.

Frames shall be passed to a bounded buffer that retains the newest available frame when processing falls behind. The buffer shall prevent unbounded memory growth and minimize recognition latency.

The module interface shall support camera initialization, frame acquisition, status reporting, and resource release. Camera device selection and retry policies shall be finalized during implementation.

3.4 Vision Module

The Vision Module shall perform hand detection and landmark extraction using a locally available pretrained model. It shall preprocess frames as required by the selected model and return detected hands with their landmark coordinates and associated detection confidence.

The module shall validate inference outputs and report model-loading or inference failures through defined error interfaces. It shall not determine the final externally recognized gesture label.

3.5 Tracking and Feature Module

The Tracking and Feature Module shall maintain the information required to interpret successive frames. Its responsibilities include:

- Associating hand detections across frames.
- Deriving finger-pose and hand-orientation features.
- Maintaining bounded motion history with timestamps.
- Calculating directional displacement and movement features.
- Computing relative spatial relationships between two hands.
- Handling missing detections and resetting invalid tracking state.

Feature outputs shall be represented using normalized coordinates or other model-independent structures where practical. Tracking identifiers and temporal history shall remain transient and shall not be written to persistent storage.

3.6 Gesture Recognition Module

The Gesture Recognition Module shall expose a stable interface for evaluating gesture candidates. It shall contain three logical recognition strategies.

Static Pose Recognizer: Uses geometric features and, where appropriate, classifiers to identify static hand configurations, including Peace Sign, Close Hand, and Open Hand.

Temporal Movement Recognizer: Uses timestamped feature history and state machines to identify Swipe Left, Swipe Right, and the four directional wave gestures involving one or two raised fingers.

Two-Hand Recognizer: Evaluates the spatial relationship and temporal interaction of two hands to detect the prayer pose or clap motion. Both patterns shall map to the single external label Join Both Hands in a Clap/Prayer Gesture.

The exact classifier, pretrained landmark model, geometric thresholds, and temporal parameters shall be selected and validated during implementation.

Figure SDP-03: Gesture Recognition Strategy Diagram

Diagram label: SDP-03 – Hybrid Gesture Recognition Strategies

Design specification: Show a common recognition interface receiving tracked hand features and branching into static pose, temporal movement, and two-hand recognition strategies. Direct all candidate results to the Recognition Decision and Event Manager. Annotate the two-hand branch to show that prayer and clap patterns produce the same external gesture identifier.

3.7 Recognition Decision and Event Manager

This module shall validate candidate recognition results and control the emission of application-level gesture events.

Its responsibilities shall include:

- Enforcing the configured confidence threshold, defaulting to 0.7.

- Applying gesture-specific temporal or geometric validation criteria.
- Resolving conflicting candidates according to a defined recognition policy.
- Suppressing duplicate events for sustained gestures.
- Managing gesture re-arming and cooldown state where required.
- Emitting validated recognition events to the Application Controller.

Recognition shall trigger as soon as the applicable gesture criteria are satisfied with sufficient confidence. Additional temporal validation shall be applied only where required by the gesture definition. Static poses and dynamic movements shall use appropriate, potentially different validation rules.

The manager shall distinguish between an ongoing recognized gesture and a newly emitted recognition event to prevent continuous frame-by-frame detection from repeatedly triggering the same event.

3.8 Configuration Module

The Configuration Module shall load and validate local application settings and provide them through a defined configuration interface.

Settings shall include the default confidence threshold of 0.7 and any subsequently defined camera, performance, or recognition parameters. Invalid or missing values shall be handled through documented defaults or explicit configuration errors.

Configuration changes shall be validated before being applied to active components. The format, storage location, and exact settings schema shall be determined during implementation.

3.9 Logging and Error Management Module

This module shall provide consistent diagnostic logging and error reporting. Log entries may include timestamps, component identifiers, severity levels, error categories, and recovery actions.

The module shall exclude camera frames, raw landmark data, and sensitive user information. Logging shall not block the GUI or delay real-time processing.

Recoverable errors shall be reported to the Application Controller for retry or component recovery. Unrecoverable errors shall result in an appropriate user-facing message and controlled resource cleanup.

3.10 Domain Module

The Domain Module shall define shared data contracts, gesture identifiers, application states, recognition results, tracking structures, and error types.

It shall remain independent of GUI widgets and concrete camera or model implementations. Shared contracts shall establish consistent communication between modules and support isolated unit testing.

4. Data Design

4.1 Core Data Structures

The following logical structures shall be defined in the Domain Module. Exact Python types and field representations shall be finalized during implementation.

Data structure	Principal contents
FrameData	Image frame, capture timestamp, frame sequence identifier
HandDetection	Hand landmarks, detection confidence, tracking identifier
HandFeatures	Finger-pose features, orientation, normalized position
MotionHistory	Timestamped feature samples and tracking state
GestureCandidate	Gesture identifier, confidence score, recognition metadata
RecognitionResult	Validated gesture identifier, confidence, event status, timestamp
ApplicationStatus	Lifecycle state, camera status, processing status
PerformanceMetrics	Camera FPS, processing FPS, relevant processing timings
ErrorRecord	Component, error category, diagnostic message, recovery status
AppConfiguration	Confidence threshold and validated runtime settings

Frame and recognition structures shall be transient. Configuration and permitted diagnostic logs may be stored locally, subject to the privacy requirements.

4.2 Gesture Identifier Design

A centralized enumeration or equivalent identifier definition shall represent the ten external gesture labels. Internal recognition strategies shall map their results to these identifiers before emitting externally visible recognition events.

The prayer pose and clap motion shall share one external identifier. Internal metadata may distinguish the detected pattern when necessary, provided this distinction does not change the externally visible gesture label.

4.3 Data Validation

Interfaces shall validate required fields, confidence ranges, timestamps, landmark dimensions, and tracking consistency. Invalid or incomplete inference outputs shall be rejected or handled safely without terminating the application. Confidence values shall be normalized to a documented range before threshold comparison.

5. Processing and Concurrency Design

5.1 Worker Allocation

The Qt main thread shall handle GUI rendering, input events, and display-state updates. Worker threads shall handle camera acquisition and vision processing, with additional separation where required by profiling and the selected model runtime.

The design shall avoid direct manipulation of GUI objects from worker threads. Inter-thread communication shall use signals, slots, thread-safe queues, or equivalent mechanisms with explicit ownership rules.

5.2 Frame Processing Pipeline

Figure SDP-04: Runtime Processing Sequence Diagram

Diagram label: SDP-04 – Camera-to-GUI Processing Sequence

Design specification: Show the Camera Worker, Frame Buffer, Vision Worker, Tracking and Feature Module, Gesture Recognition Module, Recognition Decision and Event Manager, Application Controller, and GUI as participants. Represent frame acquisition, latest-frame retrieval, landmark extraction, feature calculation, candidate generation, validation, event publication, and GUI update in sequence. Include an alternate path for invalid detections or processing errors.

5.3 Buffering and Performance

The frame buffer shall be bounded. When full, obsolete frames shall be discarded in favor of the latest available frame. Processing metrics shall use timestamps and completed-frame counts to calculate camera FPS and processing FPS consistently.

The implementation shall measure frame acquisition, inference, and end-to-end recognition latency during testing. Numeric performance targets shall be established through prototyping rather than assumed at design time.

5.4 Synchronization and Resource Ownership

Each worker shall own and release the resources it initializes unless ownership is explicitly transferred. Shared state shall be minimized. Access to mutable tracking history and recognition state shall be confined to the owning processing context or protected through suitable synchronization.

Shutdown shall stop new frame acquisition, signal workers to terminate, wait for their completion, and release camera and model resources safely. Blocking waits on the GUI thread shall be limited to controlled shutdown operations.

6. Recognition Logic Design

6.1 Recognition Evaluation

For each processed frame, the recognition pipeline shall generate hand features and evaluate applicable recognition strategies. Each strategy shall return zero or more candidates containing an external gesture identifier, confidence score, and required validation metadata.

The Recognition Decision and Event Manager shall reject candidates that fail confidence or gesture-specific validation criteria. If multiple candidates qualify, a deterministic conflict-resolution policy shall select the applicable result or reject ambiguous candidates.

6.2 Static Gestures

Static pose recognition shall evaluate the current hand configuration using geometric rules and classifiers where appropriate. Pose stability and confidence requirements shall be determined through empirical testing.

6.3 Directional Movements

Directional movement recognition shall evaluate timestamped feature history rather than isolated frames. State machines shall track gesture initiation, direction, completion, and reset conditions. Motion history shall be bounded, and stale or interrupted tracking data shall not be treated as valid movement evidence.

6.4 Two-Hand Gesture

The Two-Hand Recognizer shall determine whether the detected hands satisfy the criteria for the prayer pose or clap motion. The internal recognition path may differ between static contact-like configurations and dynamic hand convergence. Both paths shall return the same external gesture identifier.

6.5 Event Re-Arming

After emitting a recognition event, the manager shall apply the appropriate duplicate-suppression and re-arming policy. A sustained static pose shall not repeatedly emit the same event on every frame. A subsequent valid occurrence shall be permitted after the defined release, cooldown, or state-reset condition is satisfied.

Figure SDP-05: Recognition Decision and Event State Machine

Diagram label: SDP-05 – Recognition Validation and Event Re-Arming

Design specification: Show candidate evaluation, confidence validation, gesture-specific validation, event emission, active-gesture tracking, and re-arming. Include rejection paths for low-confidence and invalid candidates, and transitions that prevent duplicate events until the appropriate reset condition is satisfied.

7. Interface Design

7.1 Internal Interfaces

The following logical interfaces shall separate module responsibilities:

Interface	Purpose
Camera Interface	Initialize camera, acquire frames, report status, release resources
Vision Interface	Detect hands and extract landmarks from a frame
Feature Interface	Produce tracked hands and derived pose, motion, and spatial features
Recognition Interface	Evaluate features and return gesture candidates
Decision Interface	Validate candidates and emit recognition events
Configuration Interface	Provide validated settings to consuming modules
Logging Interface	Record diagnostic events and error information
Controller-to-GUI Interface	Publish preview, recognition, lifecycle, and performance updates

Concrete method signatures, signal definitions, and data-transfer formats shall be specified in the implementation design.

7.2 External Interfaces

The application shall use the local camera through supported operating-system and OpenCV interfaces. It shall not depend on external web services, cloud inference APIs, remote databases, or network-based model repositories during runtime.

User interaction shall occur through the PySide6 desktop interface. Installation and packaging shall supply the required runtime dependencies and model files locally.

8. Error Handling and Recovery Design

Errors shall be detected at component boundaries and translated into defined application error categories.

Failure condition	Required response
Camera unavailable	Report the error and allow safe retry or shutdown
Camera capture interruption	Report status and attempt recovery where appropriate
Model-loading failure	Prevent recognition startup and notify the user
Inference exception	Record a sanitized diagnostic entry and recover or stop safely
Invalid configuration	Apply a documented safe default or report a configuration error
Worker-thread failure	Notify the controller and coordinate recovery or shutdown
Invalid recognition output	Reject the output without emitting a recognition event
GUI communication failure	Preserve thread safety and report the failure
Shutdown interruption	Attempt bounded cleanup and release owned resources

Recovery operations shall not create duplicate workers, leave the camera open, or allow invalid recognition state to persist. Logging failures shall not cause uncontrolled application termination.

9. Security and Privacy Design

The application shall enforce offline runtime operation through its dependency and deployment design. All recognition inference shall execute locally.

Camera frames, landmarks, motion histories, and recognition intermediates shall remain in memory and shall not be written to files, databases, or logs. No telemetry or external transmission shall be implemented.

Local configuration and diagnostic logs shall be restricted to necessary operational information. Dependencies and pretrained model files shall be packaged or installed through a controlled development and deployment process. The implementation shall be reviewed to ensure that third-party libraries do not introduce unintentional network access or sensitive-data persistence.

10. Testing Design

10.1 Unit Testing

Unit tests shall validate geometric feature calculations, landmark validation, motion state machines, two-hand spatial calculations, confidence enforcement, duplicate suppression, event re-arming, configuration validation, and application state transitions.

10.2 Integration Testing

Integration tests shall verify camera-frame delivery, vision-to-feature data flow, recognition strategy integration, decision-manager event publication, controller coordination, and thread-safe GUI updates.

Where physical hardware is unnecessary, camera frames and model outputs may be replaced with controlled test inputs. Test fixtures shall avoid persistent storage of sensitive camera data.

10.3 System and Platform Testing

System tests shall verify end-to-end recognition, camera access, startup and shutdown, error recovery, performance indicators, and packaging on Windows, macOS, and Linux.

Recognition tests shall cover all ten external gesture labels under varied hand positions, orientations, lighting conditions, movement speeds, and detection interruptions. Prayer poses and clap motions shall both produce the same required external label.

10.4 Concurrency and Failure Testing

Tests shall cover dropped frames, processing overload, concurrent shutdown, worker exceptions, repeated camera failures, model-loading errors, and recovery from missing hand detections. The GUI shall remain responsive during normal processing and recoverable failures.

10.5 Performance and Acceptance Testing

Performance evaluation shall measure camera FPS, processing FPS, end-to-end latency, resource usage, and recognition accuracy. Acceptance thresholds shall be established through prototyping and documented separately before release.

11. Implementation and Deployment Considerations

Implementation shall proceed by establishing domain contracts and configuration, implementing camera acquisition and vision processing, integrating tracking and recognition strategies, adding decision and event management, and connecting the GUI and operational error handling.

Deployment shall include compatible Python runtime dependencies, PySide6, OpenCV, the selected pretrained models, and required model assets. Packaging shall be verified independently on each supported operating system. Runtime startup shall not require downloading dependencies or models.

The precise model selection, classifier implementation, temporal thresholds, camera selection policy, concurrency configuration, packaging mechanism, and quantitative performance targets remain subject to prototyping and validation.

12. Design Traceability

The following table links the major architectural requirements to their corresponding software design elements.

Requirement	Design element	Verification
Ten predefined gestures	Gesture Recognition Module and Domain Module	Gesture-level unit and system tests
Prayer and clap share one label	Two-Hand Recognizer and centralized identifiers	Two-hand integration tests
Default confidence threshold of 0.7	Configuration Module and Decision Manager	Configuration and decision tests
Immediate triggering after valid detection	Decision Manager	Event latency and trigger-policy tests
Offline execution	Local model assets and runtime dependencies	Network-isolation testing
Responsive GUI	Main-thread presentation and worker processing	Concurrency and responsiveness tests
Bounded processing latency	Latest-frame-wins buffer	Processing-load tests
No persistent camera data	In-memory processing and restricted logging	Privacy and storage inspection
Cross-platform support	Camera abstraction and deployment configuration	Platform-specific system tests
Duplicate event suppression	Decision Manager and re-arming state	Recognition state-machine tests

13. Conclusion

This Software Design Plan defines the component-level design and operational behavior of Version 1 of the Offline Gesture Recognition Application. It establishes the module boundaries, data contracts, recognition workflow, threading model, event validation policy, error handling, privacy controls, and testing approach required to implement the approved architecture.

The design preserves the separation between the GUI, camera acquisition, vision processing, gesture recognition, and event management. Implementation-specific parameters shall be finalized through prototyping and verified against documented functional, performance, cross-platform, and privacy requirements.